ALPRO TECHNOLOGY SPA

Memoria del Proyecto Final de Máster

Alumno: cuenta de GitHub yondaimejin4

Correo: el registrado en Academia (no figura en el repositorio)

Convocatoria: octubre de 2026

Repositorio: https://github.com/yondaimejin4/tareasConquerblocks
Carpeta del proyecto: proyecto final master full stack
Aplicación local: http://127.0.0.1:8000/
Aplicación pública: pendiente de publicar en Render o Railway. El código y render.yaml ya están preparados.
Vídeo: guion en el apartado 12. La grabación la hace el alumno.

Usuarios de prueba

Administrador: admin / Admin123 Panel: /admin/
Cliente: cliente_demo / Cliente123 Portal: /portal/login

Índice

- 1. Definición del problema
- 2. Aportación y eficiencia
- 3. Tecnologías y justificación
- 4. Tipos de usuarios
- 5. Casos de uso
- 6. Seguridad y protección de datos
- 7. Arquitectura y modelo de datos
- 8. Estructura del repositorio
- 9. Despliegue y pruebas
- 10. Uso de herramientas de IA
- 11. Limitaciones, trabajo futuro y conclusiones
- 12. Guion del vídeo y de la defensa
- 13. Lista de comprobación de la guía v2

1. Definición del problema

Contexto

ALPRO TECHNOLOGY SPA presta desarrollo web, automatización y consultoría a empresas. Antes de esta herramienta, la captación comercial dependía de WhatsApp, correo y llamadas. El catálogo de servicios no vivía en un solo sitio y el seguimiento de cada encargo se anotaba fuera del sistema, sin que el cliente viera en qué fase iba su proyecto ni el estado del pago.

Carencias

- No había un catálogo digital actualizado y consultable.
- Las solicitudes no quedaban en una base de datos con fecha, empresa y mensaje.
- El cliente no tenía un acceso propio para ver el avance ni aportar ideas.
- El equipo no distinguía, en la misma herramienta, notas internas y mensajes visibles al cliente.

Impacto y público

El retraso en responder y la falta de historial hacen perder oportunidades y dan una imagen poco ordenada. El público es doble: la empresa que busca una consultoría (visitante) y el cliente que ya encargó un trabajo. El tercer actor es el administrador de ALPRO.

Solución

Una aplicación web con landing en React que lee el catálogo por API, un formulario de consultoría, un portal de cliente y Django Admin. El administrador actualiza el estado del proyecto, el pago y un comentario. El cliente lo ve al recargar su solicitud. El pago no se cobra con tarjeta: se registra a mano. Ese límite es deliberado para mantener un MVP coherente.

2. Aportación y eficiencia

La herramienta convierte mensajes sueltos en registros. Un visitante completa nombre, correo, empresa y necesidad, y el servidor valida el correo antes de guardar. El equipo deja de copiar el mismo dato en una hoja y en el chat. El cliente deja de preguntar por WhatsApp en qué va el trabajo: abre el portal y lee el estado, el comentario del equipo y si el pago está pendiente, parcial o saldado.

Ejemplo concreto. El cliente cliente_demo envía que necesita una web y automatizar el seguimiento. El administrador marca «En desarrollo», escribe que se está diseñando la estructura y anota que falta el abono del 50 por ciento. Esas tres frases aparecen en el detalle del portal. La nota interna («llamar el viernes») no sale de Admin. Se ahorra una ida y vuelta y se evita filtrar comentarios de equipo.

El catálogo también deja de ser un PDF. Cambiar un servicio destacado en Admin cambia lo que React muestra en la siguiente carga, porque la landing no tiene los servicios escritos a fuego: los pide a la API.

3. Tecnologías y justificación

- Django 6.0.5. Modelos, migraciones, admin, sesiones y plantillas. Es el backend exigido por el máster.
- Django REST Framework 3.17.1. La API JSON que consume React, con validación en serializers.
- SQLite en local y PostgreSQL en producción mediante dj-database-url 2.3.0 y psycopg2-binary 2.9.10.
- React 19 y React Router 7, empaquetados con Vite 8. La landing y el portal son las vistas que leen la API.

- Pillow 11.1.0 para la imagen opcional de un aporte.
- WhiteNoise 6.9.0 y Gunicorn 23.0.0 para estáticos y el proceso de producción.
- django-cors-headers 4.9.0 por si el frontend y la API se separan en otro origen durante el desarrollo.
- Git y GitHub. El remoto es el repositorio público indicado en la portada.

Docker no se usa. No aporta al MVP y la guía lo marca como opcional. El despliegue previsto es un servicio web de Render con el blueprint render.yaml: build.sh instala dependencias, compila React, recoge estáticos, migra y siembra el catálogo.

4. Tipos de usuarios

Visitante

Entra sin cuenta. Ve la landing, el catálogo y los destacados. Envía el formulario de consultoría. Esa solicitud se guarda sin usuario asociado. No ve el portal ni el admin.

Cliente

Tiene un usuario de Django que no es staff. Puede registrarse en /portal/registro o usar una cuenta creada por el administrador. Inicia sesión en /portal/login. Crea solicitudes, lee solo las suyas, envía aportes de texto o imagen y consulta estado, pago y comentario del equipo.

Administrador

Es superusuario. Entra en /admin/ con admin / Admin123 en el entorno de demostración. Crea clientes, edita servicios, abre cada solicitud, ve los aportes y cambia estado, comentario visible, pago y notas internas.

5. Casos de uso

CU-01. Consultar servicios (visitante, React)

Flujo: abre la raíz, React llama a GET /api/services/ y GET /api/services/featured/, y pinta tarjetas. Resultado correcto: el aviso «Conectado al servidor» y el listado. Error: si el backend no responde, la página muestra el fallo y no inventa servicios.



Figura 1. Landing. Los datos del catálogo vienen de la API, no están fijos en React.



Figura 2. Servicio destacado y catálogo completo, ambos leídos del mismo backend.

CU-02. Pedir una consultoría (visitante)

Flujo: completa nombre, correo, empresa opcional y mensaje, y envía POST /api/contact/. Resultado: HTTP 201 y mensaje de gracias; la fila aparece en Admin. Error: correo inválido, HTTP

400, y el texto del error se muestra en el formulario. Si el administrador tiene su sesión abierta en otra pestaña, el lead no se le asigna: esa vista no usa la cookie del admin.

CU-03. Entrar al portal (cliente)

Flujo: usuario y contraseña en /portal/login. Resultado: saludo «Bienvenido» más el nombre de usuario y la lista de sus solicitudes. Error: credenciales malas, mensaje de usuario o contraseña incorrectos, sin crear sesión.



Figura 3. Acceso del cliente. La sesión es distinta de la del administrador.



Figura 4. Lista del cliente con estado, pago y aviso de pendiente.

CU-04. Ver el avance y aportar una idea (cliente)

Flujo: abre el detalle. Lee estado, comentario del equipo y pago. Puede añadir un texto y una imagen. Resultado: el aporte queda ligado a su usuario y a esa solicitud, y el administrador lo ve en el inline. Error: si pide el identificador de una solicitud de otro cliente, la API responde 404.



Figura 5. Detalle: comentario del equipo, pago parcial y aporte del cliente.

CU-05. Gestionar la solicitud (administrador)

Flujo: entra en /admin/, abre Solicitudes de clientes, cambia el estado del proyecto, el comentario visible y el estado de pago, y guarda. Resultado: el cliente lo ve al recargar. Las notas internas no viajan en el JSON. Error típico: guardar sin completar un campo obligatorio del modelo; Django Admin lo señala en el formulario.

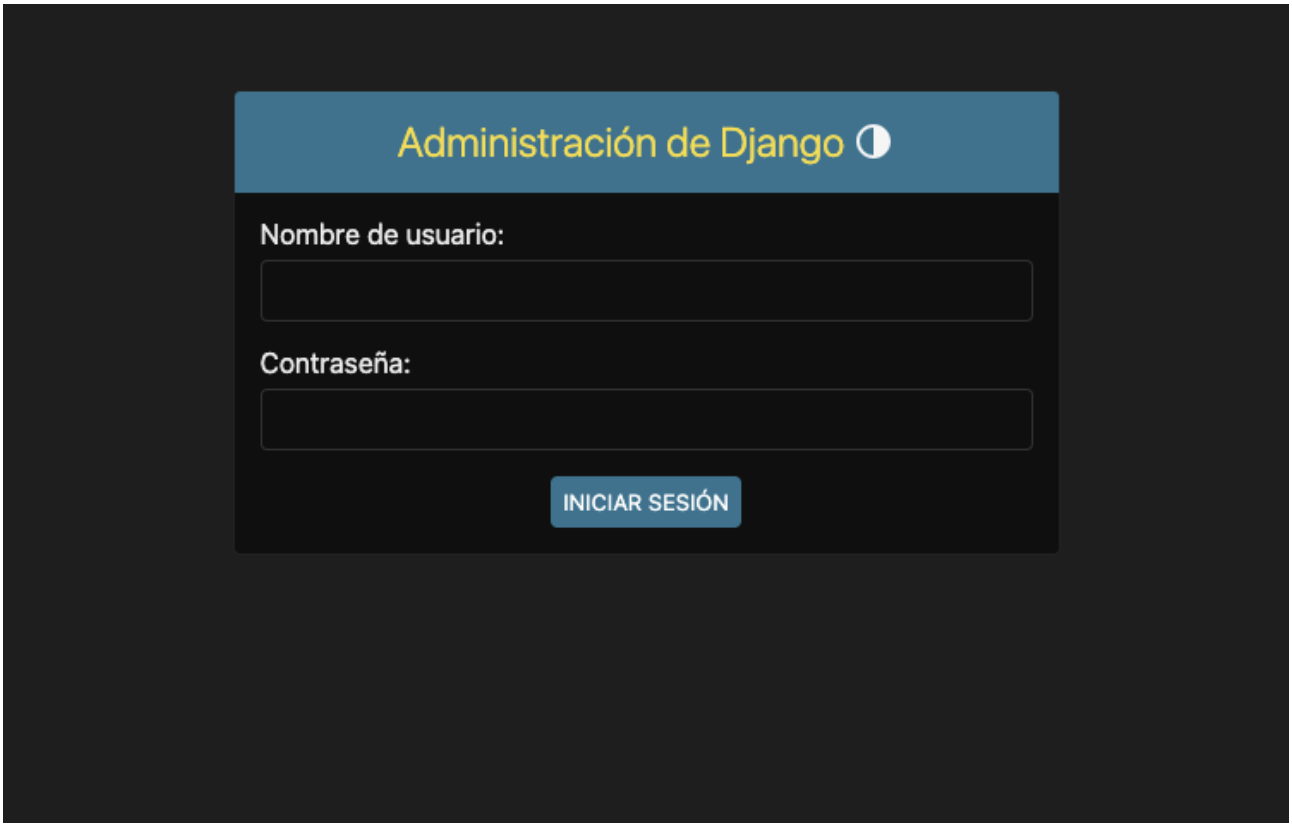


Figura 6. Acceso al panel. Usuario de prueba: admin / Admin123.

CU-06. Mantener el catálogo (administrador)

Crea o edita un Service y marca is_featured. Al recargar la landing, React vuelve a pedir la API y muestra el cambio.

6. Seguridad y protección de datos

- Autenticación del admin con django.contrib.auth. Solo is_staff entra al panel.
- Autenticación del portal con la cookie portal_sessionid, no con la cookie sessionid del admin.
- Contraseñas con el hasher PBKDF2 de Django. No se guardan en claro ni se devuelven en la API.
- Validación de entrada en serializers: correo, longitud y campos obligatorios.
- CSRF en el admin y en los POST del portal (cookie csrftoken y cabecera X-CSRFToken).
- El formulario público no autentica, para no exigir CSRF de una sesión de staff ni mezclar usuarios.
- Aislamiento: cada cliente solo lista y modifica ContactLead.user igual a él.

- XSS: React escapa el texto; las plantillas de Django también.
- Secretos por entorno: SECRET_KEY, DEBUG, ALLOWED_HOSTS, CORS y DATABASE_URL. El archivo .env no se versiona.
- HTTPS y DEBUG=False corresponden al despliegue. En local DEBUG está activo a propósito.
- Imágenes de aportes en media/, no ejecutables. Pillow valida que el archivo sea imagen.
- Copias de seguridad: en producción las asume el PostgreSQL del proveedor. En local la base es backend/db.sqlite3, ignorada por Git.

Comprobación recomendada por la guía: no hay claves en el historial; CORS solo abre los orígenes listados; requirements.txt fija versiones. No se ejecutó un auditor externo de dependencias en esta entrega.

7. Arquitectura y modelo de datos

El navegador carga la SPA desde el mismo origen que la API. Django lee frontend/dist y responde index.html en las rutas que no son /api, /admin, /django ni /media. Así las cookies funcionan sin CORS.

Navegador React --> HTTP JSON --> Django y DRF --> SQLite o PostgreSQL. El admin usa la sesión clásica. El portal usa portal_sessionid. WhiteNoise sirve el CSS del admin.

Entidades

- User (Django): username, email, nombre, contraseña hasheada, is_staff, is_superuser.
- Service: title, short_description, details, is_featured, created_at. No depende de un usuario.
- ContactLead: user opcional (SET_NULL), name, email, company, message, status, payment_status, pending_summary, team_comment, admin_notes, fechas.
- ClientContribution: lead (CASCADE), user (CASCADE), message, attachment opcional, created_at.

Un usuario tiene muchas solicitudes. Una solicitud tiene muchos aportes. Borrar el usuario no borra la solicitud: el campo user queda vacío. Borrar la solicitud sí borra sus aportes.

Estados

Proyecto: recibida, en revisión, en desarrollo, entregada. Pago: pendiente, pago parcial, pagado. team_comment y pending_summary son visibles en el portal. admin_notes no está en el serializer del cliente.

8. Estructura del repositorio

En la raíz de GitHub hay un índice, la carpeta de este proyecto y entregas/spa-beauty, que es una práctica anterior. Dentro de «proyecto final master full stack» están backend/, frontend/, docs/, scripts/, requirements.txt, build.sh y render.yaml. El código de la API está en backend/core. Las pantallas React están en frontend/src/pages/LandingPage.jsx y Portal.jsx. El cliente HTTP está en frontend/src/api/client.js.

scripts/dev.sh activa el flujo local: migraciones, seed_demo, seed_admin, npm run build y runserver

en el puerto 8000.

9. Despliegue y pruebas

Local: desde la carpeta del proyecto, source .venv/bin/activate y ./scripts/dev.sh. Quedan 16 servicios de demostración y el administrador admin / Admin123. El cliente de prueba de esta memoria es cliente_demo / Cliente123.

Producción prevista: Render, con PostgreSQL, DEBUG=False, SECRET_KEY generada y ALLOWED_HOSTS del dominio. build.sh instala Python, compila el frontend, collectstatic, migrate y seed_demo. El proceso es Gunicorn. A la fecha de esta memoria la URL pública todavía no está dada de alta: hace falta la cuenta del alumno en Render o Railway para pulsar el despliegue. El archivo render.yaml ya describe el servicio.

Pruebas automatizadas: python manage.py test core. En la última ejecución local pasaron 10 tests: listado y destacados de servicios, alta de contacto, contacto con sesión de admin abierta, correo inválido, registro y listado propio, aislamiento entre clientes, sesión de admin que no entra en el portal, comentario y pago visibles para el cliente, y login de una cuenta creada como en el admin.

Errores que aparecieron y cómo se cerraron

- La landing quedaba en blanco por una ruta de CSS incorrecta al separar las páginas. Se corrigió el import.
- Dos puertos (Vite y Django) mezclaban CORS y cookies. Se pasó a un solo origen: Django sirve el build de React.
- Con el admin abierto, el formulario público devolvía 403 por CSRF. La vista de contacto dejó de usar la sesión de staff.
- Las solicitudes del portal salían a nombre del admin. Se creó la cookie portal_sessionid.

10. Uso de herramientas de IA

Se usó el asistente de programación Cursor para proponer código, depurar errores de sesión y CSRF, y redactar borradores de documentación. La definición del problema (una empresa real que necesita catálogo, solicitudes y seguimiento), el modelo de datos y la decisión de separar las dos sesiones se revisaron y se probaron en la aplicación.

Cómo se validó el resultado: los 10 tests automáticos, el recorrido manual de landing, portal y admin, y la lectura del código de models.py, views.py, portal_auth.py y LandingPage.jsx. Una declaración honesta no sustituye la defensa: en la videollamada hay que poder modificar y explicar cualquier parte.

11. Limitaciones, trabajo futuro y conclusiones

No hay pasarela de pago, ni correo transaccional, ni doble factor, ni auditoría formal de dependencias. El catálogo y los textos de estado están en castellano sin tildes en varios rótulos de interfaz, por consistencia con el primer diseño. La URL pública aún no está en línea.

Más adelante: publicar en Render con HTTPS, aviso por correo al crear una solicitud, pasarela de

pago real y copias automáticas de PostgreSQL. No amplía el alcance antes de la defensa.

Conclusión. El proyecto resuelve un problema concreto con un flujo de datos real: React muestra lo que Django guarda, el cliente solo ve lo suyo y el administrador controla el estado sin exponer las notas internas. Es un MVP pequeño y explicable, que es lo que pide la guía.

12. Guion del vídeo (máximo 5 minutos) y de la defensa

El vídeo lo graba el alumno (Loom, OBS o el móvil) mostrando la pantalla. Enlace sin pedir permiso. Texto sugerido, en voz alta:

- 0:00. Soy el alumno del máster. El problema es que ALPRO recibía encargos por canales sueltos, sin catálogo ni seguimiento.
- 0:40. Esta es la landing. Estos servicios no están escritos en React: salen de GET /api/services/. Lo enseño en la pestaña Red.
- 1:30. Envío una consultoría. Django valida y guarda un ContactLead.
- 2:10. Entro al portal como cliente_demo. Veo el estado, el pago y el comentario que escribió el admin.
- 3:10. En /admin/ cambio el estado. Tecnologías: Django 6, DRF, React 19, Vite, SQLite en local y PostgreSQL en producción.
- 4:20. Cierro: el cliente y el admin no comparten sesión, para no mezclar solicitudes.

Defensa de 30 minutos. Tener abiertos la URL (cuando exista), el editor con el servidor local y GitHub. Minutos 0 a 3: problema y usuarios. 3 a 12: demo. 12 a 18: models.py y el fetch de la landing. 18 a 24: las dos cookies y el test que lo demuestra. 24 a 30: seguridad y límites. Si piden un cambio en directo, conviene un texto visible o un choice de estado, no rehacer la autenticación.

13. Lista de comprobación de la guía v2

- Repositorio público con README, dependencias versionadas y sin .env: hecho.
- Backend Django con modelos relacionados, migraciones, vistas, plantillas (/django/), formularios validados y dos roles: hecho.
- React muestra datos reales de la API: hecho en local. Hay que repetirlo en la URL pública.
- Tests automáticos: 10, en verde.
- Memoria en PDF con texto seleccionable y los apartados del punto 3: este archivo.
- URL HTTPS con DEBUG desactivado: pendiente de la cuenta de despliegue del alumno.
- Vídeo de menos de 5 minutos: pendiente de grabar con el guion del apartado 12.
- En Academia hay que pegar repo, URL, este PDF, el vídeo y las credenciales admin / Admin123 y cliente_demo / Cliente123.

Nombre y correo de la portada: si Academia tiene otro nombre distinto de la cuenta yondaimejin4, sustituir la línea de la portada antes de subir el PDF. No inventé un correo que no está en el repositorio.

14. Diccionario de datos

Service.title es el nombre visible, máximo 120 caracteres. short_description cabe en la tarjeta (200).

details es el texto largo que se muestra al pulsar Ver detalle. is_featured decide si entra en GET /api/services/featured/. created_at lo rellena la base al insertar.

ContactLead.user apunta al cliente cuando la solicitud nace en el portal. Si nace en el formulario público, queda vacío. name y email se copian del formulario o, en el portal, del nombre completo del usuario y de su correo. company puede ir vacío. message es el encargo. status y payment_status usan códigos cortos en la base (received, in_progress, pending, partial, paid) y etiquetas en español que calcula Django con get_status_display. pending_summary es una frase corta de cobro. team_comment es el texto libre del equipo. admin_notes no se serializa hacia el cliente. updated_at cambia en cada guardado.

ClientContribution.message es obligatorio. attachment acepta imagen o nada. La ruta de disco es media/contributions/año/mes/. En la API el cliente recibe attachment_url absoluta, no la ruta interna.

15. Recorrido del código, archivo por archivo

backend/core/models.py define las tres entidades del negocio y el usuario lo aporta Django. Las migraciones 0001 a 0004 crean la tabla, los campos de seguimiento, el comentario del equipo y las etiquetas en español. No se editan a mano las migraciones ya aplicadas: un cambio de modelo genera otra migración.

backend/core/serializers.py traduce JSON a datos válidos y al revés. ServiceSerializer es de solo lectura para el catálogo. ContactLeadCreateSerializer exige nombre, correo y mensaje. RegisterSerializer rechaza usuario o correo repetidos y pasa la contraseña por los validadores de Django. ProjectRequestSerializer incluye status_label, payment_status_label, team_comment y la lista de aportes. No incluye admin_notes.

backend/core/views.py agrupa la API. HealthAPIView sirve para comprobar que el proceso vive. ServiceListAPIView y FeaturedServiceListAPIView son listas públicas. ContactLeadCreateAPIView desactiva la autenticación de sesión. Register, login y logout del portal escriben o borran portal_sessionid. MeAPIView dice si hay cliente, no si hay admin. MyProjectRequestListCreateAPIView filtra por request.user, que aquí es el usuario de la cookie del portal.

backend/core/portal_auth.py abre una SessionStore propia, guarda el id del usuario y devuelve la clave de sesión. get_portal_user lee la cookie y busca el User. Si la cookie no existe o el usuario fue borrado, devuelve None. authentication.py adapta eso a DRF: authenticate devuelve (user, None) o None.

backend/core/admin.py muestra la solicitud en bloques: datos del cliente, estado del proyecto con el comentario, estado de pago y notas internas plegadas. Los aportes van en un inline de solo lectura. En la lista se puede cambiar estado y pago sin abrir la ficha. admin_users.py registra el alta de usuarios con correo y contraseña, sin convertirlos en staff.

frontend/src/pages/LandingPage.jsx, al montarse, llama a fetchServices. El formulario usa apiFetch hacia /api/contact/ después de ensureCsrf. frontend/src/pages/Portal.jsx pide /api/auth/me/ al cargar. Si no hay sesión, manda al login. Si la hay, pinta el saludo y las rutas de solicitudes. frontend/src/api/client.js centraliza la cookie CSRF y credentials: include, que es lo que permite

enviar portal_sessionid.

backend/core/spa_views.py resuelve frontend/dist/index.html para cualquier ruta que no haya coincidido antes. Por eso /portal/solicitudes/12 lo atiende React Router en el navegador, no una vista Django por cada pantalla.

16. Qué responder si piden un cambio en directo

Cambio seguro: en LandingPage.jsx, el titular del hero. Se guarda el archivo, se vuelve a ejecutar npm run build dentro de frontend y se recarga el puerto 8000. Si el servidor de desarrollo de Vite está activo, el cambio se ve sin build.

Otro cambio seguro: añadir un valor al estado del proyecto en ContactLead.Status, crear la migración y mostrarlo en el admin. No hace falta tocar React si el portal ya pinta status_label, porque la etiqueta sale del servidor.

Cambio que no conviene improvisar en la llamada: unir otra vez las dos cookies, quitar el CSRF o devolver admin_notes en el serializer. Cualquiera de los tres rompe una decisión ya explicada.

17. Instalación que se puede repetir delante del profesor

Requisitos: Python 3.13 y Node 20. En la carpeta del proyecto: python3 -m venv .venv, source .venv/bin/activate, pip install -r requirements.txt, cd frontend && npm install && cd .., ./scripts/dev.sh. El script migra, carga 16 servicios, asegura el admin y compila React. La comprobación rápida es abrir /api/health/ y ver ok en true junto al número de servicios.

Variables de entorno de ejemplo están en .env.example: SECRET_KEY, DEBUG, ALLOWED_HOSTS y CORS_ALLOWED_ORIGINS. En local se puede arrancar sin .env porque settings.py trae valores de desarrollo. Esos valores de desarrollo no deben usarse cuando DEBUG pase a False.

18. Los diez tests, uno por uno

test_list_services_returns_json comprueba que GET /api/services/ responde 200 y devuelve el título guardado.

test_featured_services_only_returns_featured crea un servicio no destacado y verifica que el endpoint de destacados no lo incluye.

test_create_contact_lead_success envía un JSON válido y espera 201 y una fila en ContactLead.

test_create_contact_lead_invalid_email envía un correo roto y espera 400 y cero filas nuevas.

test_create_contact_with_admin_session_cookie_succeeds deja una sesión de superusuario activa y aun así el POST público debe crear el lead con user vacío.

test_register_and_list_own_requests registra un usuario por la API, crea una solicitud y comprueba que el listado devuelve solo esa, en estado recibida.

test_client_cannot_see_other_users_request crea dos clientes y comprueba que el detalle ajeno responde 404.

test_admin_django_session_does_not_bleed_into_portal inicia sesión de admin al estilo Django y

comprueba que /api/auth/me/ del portal sigue anónimo hasta que entra el cliente. La solicitud creada después queda con el id del cliente, no del admin.

test_client_sees_admin_team_comment_and_contributions escribe estado, pago, pendiente y comentario, añade un aporte y comprueba que el detalle del cliente trae esos campos y el texto del aporte.

test_admin_created_client_can_login crea el usuario como lo haría el alta del admin y entra por /api/auth/login/.

19. Preguntas de defensa con la respuesta ya dicha en voz alta

¿Dónde está el flujo de datos?

En la landing, el useEffect llama a fetchServices. Esa función hace dos fetch. Django ejecuta el queryset de Service, el serializer lo convierte en lista JSON y React hace setState. No hay una copia del catálogo dentro del JavaScript.

¿Por qué 404 y no 403 cuando un cliente pide una solicitud ajena?

Porque el queryset ya filtra por usuario. La fila existe, pero para ese cliente es como si no existiera. Así no se confirma que el identificador pertenece a otra persona.

¿Qué pasa con la imagen del aporte?

El navegador manda multipart con el campo attachment. ContributionCreateSerializer usa ImageField. El archivo se guarda con el storage de Django. La respuesta incluye la URL para abrirla en otra pestaña.

¿El administrador puede crear el cliente?

Sí. En Usuarios, Añadir usuario, se piden usuario, correo y contraseña dos veces. save pone is_staff e is_superuser en falso. Esa persona entra en el portal con esas credenciales. En la ficha del usuario se listan sus solicitudes.

¿Qué no debo afirmar en la defensa?

No debo decir que el pago está integrado con un banco, que el sitio ya está en HTTPS si todavía no se ha publicado, ni que el formulario público identifica al visitante. Si no hay sesión de portal, el lead queda anónimo y el equipo lo ve igual en Admin.

20. Orden exacto para entregar en Academia

- Subir este PDF: docs/Memoria_PFM_ALPRO.pdf. Si el nombre o el correo de Academia son otros, cambiar la portada y volver a generar con python3 docs/generar_memoria_pdf.py.
- Pegar el repositorio: https://github.com/yondaimejin4/tareasConquerblocks y decir que el código está en la carpeta proyecto final master full stack.
- Publicar la app, pegar la URL HTTPS y comprobar que la landing muestra los 16 servicios.
- Grabar el vídeo con el apartado 12, subirlo a Drive o YouTube en modo enlace público y pegar la URL.

- En el comentario de la entrega escribir: admin / Admin123 y cliente_demo / Cliente123.